
Clignotement de LED et Feu de Signalisation

Simulation d'un feu tricolore par gestion d'une machine à états temporisée

Darix SAMANI SIEWE
Rapport de projet – Systeme a Microcontrolleurs

1^{er} juillet 2026

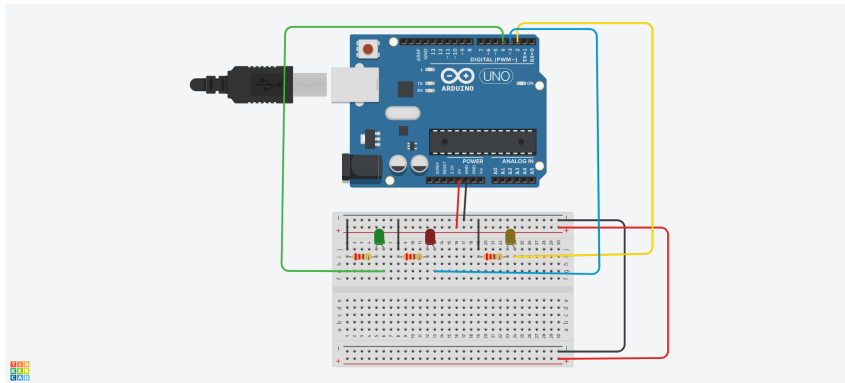


FIGURE 1 – Vue du montage du projet (capture Tinkercad ou photo réelle à insérer ici).

Table des matières

1	Introduction	2
2	Description du Projet	2
3	Composants Utilisés	2
3.1	Composants du cahier des charges	2
3.2	Composants utilisés dans la simulation Tinkercad	3
4	Notions Électroniques Clés	3
5	Schéma du Montage	4
6	Code Arduino	4
7	Analyse et Discussion du Code	5
8	Ressources Complémentaires	6
9	Conclusion	6

1 Introduction

Ce rapport présente un projet d'initiation à la programmation Arduino, centré sur la **maîtrise des sorties numériques**, la **gestion des temporisations** et le pilotage de plusieurs LED pour simuler un **feu tricolore**. Bien que volontairement simple, ce projet constitue une excellente porte d'entrée vers des notions fondamentales que l'on retrouve dans pratiquement tous les projets Arduino plus avancés : commande de sorties `digitalWrite()`, gestion du temps, et structuration du comportement sous forme de **machine à états**.

L'objectif est de reproduire la séquence classique d'un feu de circulation – **vert** → **jaune** → **rouge** – en respectant des durées réalistes pour chaque phase, à l'aide de trois LED de couleurs différentes.

2 Description du Projet

Le système commande trois LED (verte, jaune, rouge) selon une **séquence cyclique** reproduisant le fonctionnement d'un feu de signalisation routier :

- **Feu vert** allumé pendant 3 secondes (circulation autorisée) ;
- **Feu jaune** allumé pendant 1 seconde (transition, signal d'alerte avant arrêt) ;
- **Feu rouge** allumé pendant 3 secondes (arrêt obligatoire) ;
- retour au **feu vert**, et répétition indéfinie du cycle.

Plutôt que d'utiliser une succession brute de `delay()` bloquants, le programme structure ce comportement sous forme de **machine à états finis** (`enum Etats`), où chaque état correspond à une couleur de feu active et à une durée associée. Le passage d'un état à l'autre est géré de façon **non bloquante** grâce à la fonction `millis()`, qui mesure le temps écoulé depuis le dernier changement d'état sans interrompre l'exécution du programme.

Objectif pédagogique : Ce projet illustre la différence fondamentale entre une temporisation **bloquante** (`delay()`) et une temporisation **non bloquante** (`millis()`), un concept central dès qu'un programme Arduino doit gérer plusieurs tâches simultanées (lecture de capteurs, communication, affichage) sans que l'une ne bloque les autres.

3 Composants Utilisés

3.1 Composants du cahier des charges

lightgray Composant	Rôle dans le projet
Arduino Uno	Microcontrôleur assurant la gestion du temps et le pilotage des sorties numériques
Breadboard	Plaque d'essai pour le câblage du montage
LED rouge (x1)	Indication « arrêt » du feu de signalisation
LED jaune (x1)	Indication « transition / attention » du feu de signalisation
LED verte (x1)	Indication « passage autorisé » du feu de signalisation
Résistances 220 Ω (x3)	Limitation du courant traversant chaque LED

TABLE 1 – Composants prévus dans le cahier des charges du projet.

3.2 Composants utilisés dans la simulation Tinkercad

lightgray Réf- rence	Quantité	Composant
U1	1	Arduino Uno R3
D1	1	LED rouge (broche 3)
D2	1	LED jaune (broche 2)
D3	1	LED verte (broche 4)
R1, R2, R3	3	Résistances 220 Ω (une par LED)

TABLE 2 – Composants effectivement câblés dans la simulation Tinkercad.

Remarque : Ce projet ne nécessite aucune substitution de composant : la simulation Tinkercad correspond exactement au cahier des charges initial (Arduino Uno, trois LED, trois résistances), ce qui en fait un excellent point de départ fiable avant un portage direct sur montage physique réel.

4 Notions Électroniques Clés

- **Sorties numériques et résistances limitatrices de courant :** chaque LED est pilotée par une broche numérique de l'Arduino configurée en sortie (OUTPUT), capable de fournir une tension de 5 V. Une résistance série (ici 220 Ω) est indispensable pour limiter le courant traversant la LED à une valeur sûre (typiquement 15–20 mA), selon la loi d'Ohm :

$$R = \frac{V_{alim} - V_{LED}}{I_{LED}}$$

où V_{LED} est la tension de seuil de la LED (environ 1,8 à 2,2 V selon la couleur) et I_{LED} le courant souhaité.

- **Fonctions digitalWrite() et delay() / millis() :** digitalWrite() permet de fixer l'état logique (HIGH/LOW) d'une broche numérique, tandis que delay() suspend l'exécution du programme pendant une durée donnée (approche bloquante) et millis() renvoie le temps écoulé depuis le démarrage du programme, permettant de construire des temporisations **non bloquantes** en comparant les écarts de temps.
- **Diagramme d'états séquentiels :** le comportement du feu tricolore peut être modélisé par un diagramme à trois états (Vert, Jaune, Rouge) reliés par des transitions temporisées et cycliques. Cette modélisation, bien qu'élémentaire ici, constitue la base conceptuelle des **machines à états finis** utilisées dans des systèmes embarqués bien plus complexes (alarmes, automates industriels, protocoles de communication).

5 Schéma du Montage

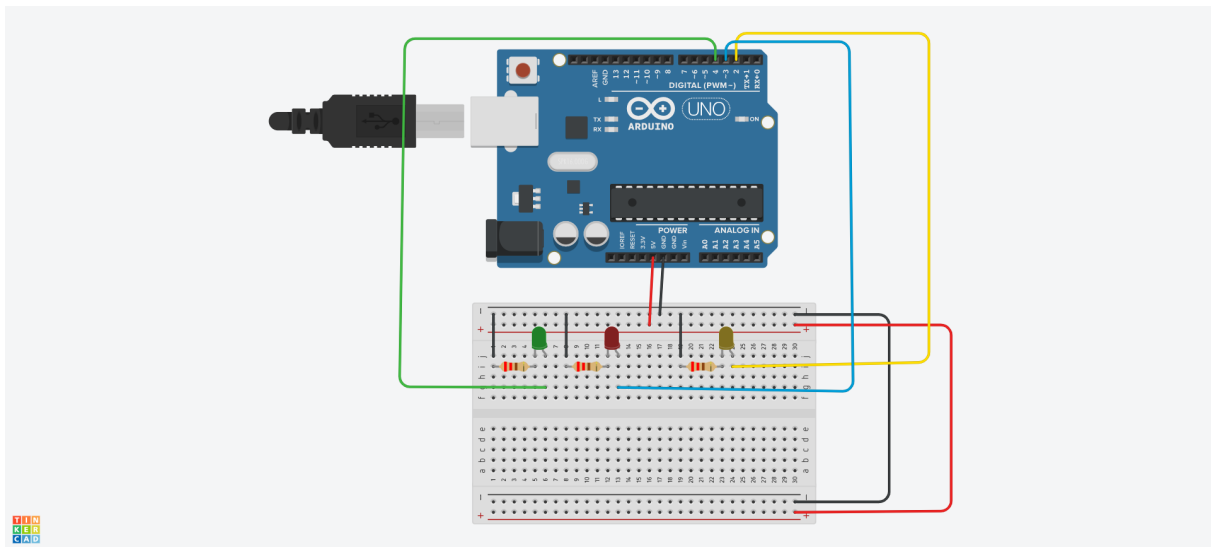


FIGURE 2 – Schéma de câblage du montage (Arduino Uno, LED rouge/jaune/verte, résistances 220 Ω).

6 Code Arduino

Le programme implémente la machine à états du feu tricolore en gestion non bloquante du temps via `millis()`, garantissant des transitions précises et un code facilement extensible.

```

1 // --- Attribution des Broches ---
2 const int pinVert = 4;
3 const int pinJaune = 2;
4 const int pinRouge = 3;
5
6 // --- Temporisations (en millisecondes) ---
7 const unsigned long DUREE_VERT = 3000; // 3 secondes
8 const unsigned long DUREE_JAUNE = 1000; // 1 seconde
9 const unsigned long DUREE_ROUGE = 3000; // 3 secondes
10
11 // --- Gestionnaire de la Machine à états ---
12 enum Etats {FEU_VERT, FEU_JAUNE, FEU_ROUGE};
13 Etats etatCourant = FEU_VERT;
14 unsigned long tempsPrecedent = 0;
15
16 void setup() {
17     // Configuration des broches en sortie
18     pinMode(pinVert, OUTPUT);
19     pinMode(pinJaune, OUTPUT);
20     pinMode(pinRouge, OUTPUT);
21
22     // tat initial : Feu Vert allum
23     rafraichirLeds(HIGH, LOW, LOW);
24 }
25
26 void loop() {
27     unsigned long tempsActuel = millis();
28     unsigned long dureeEtape = 0;
29
30     // Determination de la durée selon l'état en cours
31     if (etatCourant == FEU_VERT) dureeEtape = DUREE_VERT;
  
```

```

32  else if (etatCourant == FEU_JAUNE) dureeEtape = DUREE_JAUNE;
33  else if (etatCourant == FEU_ROUGE) dureeEtape = DUREE_ROUGE;
34
35  // V r i f i c a t i o n   d u   t e m p s   c o u l   ( S t r u c t u r e   a s y n c h r o n e )
36  if (tempsActuel - tempsPrecedent >= dureeEtape) {
37      tempsPrecedent = tempsActuel; // R i n i t i a l i s a t i o n   d u   c h r o n o m t r e
38
39      // T r a n s i t i o n   d ' e t a t   ( S e q u e n c e   c l a s s i q u e )
40      switch (etatCourant) {
41          case FEU_VERT:
42              etatCourant = FEU_JAUNE;
43              rafraichirLeds(LOW, HIGH, LOW); // A l l u m e   J a u n e
44              break;
45
46          case FEU_JAUNE:
47              etatCourant = FEU_ROUGE;
48              rafraichirLeds(LOW, LOW, HIGH); // A l l u m e   R o u g e
49              break;
50
51          case FEU_ROUGE:
52              etatCourant = FEU_VERT;
53              rafraichirLeds(HIGH, LOW, LOW); // A l l u m e   V e r t
54              break;
55      }
56  }
57 }
58
59 // F o n c t i o n   u t i l i t a i r e   p o u r   c o m m a n d e r   l e s   s o r t i e s   l o g i q u e s   d ' u n   c o u p
60 void rafraichirLeds(int v, int j, int r) {
61     digitalWrite(pinVert, v);
62     digitalWrite(pinJaune, j);
63     digitalWrite(pinRouge, r);
64 }

```

Listing 1 – Code source Arduino du feu tricolore (simulation Tinkercad).

7 Analyse et Discussion du Code

Le code peut être décomposé en quatre blocs fonctionnels principaux :

1. **Déclarations et configuration (setup)** : les trois broches pilotant les LED sont définies en constantes (`pinVert`, `pinJaune`, `pinRouge`) puis configurées en sortie via `pinMode()`. L'état initial du système est fixé au **feu vert** grâce à un appel à `rafraichirLeds(HIGH, LOW, LOW)`.
2. **Machine à états (enum Etats)** : le type énuméré `Etats` (`FEU_VERT`, `FEU_JAUNE`, `FEU_ROUGE`) rend le code particulièrement lisible, en remplaçant des constantes numériques arbitraires par des noms explicites. La variable `etatCourant` mémorise en permanence la phase active du feu.
3. **Temporisation non bloquante (loop)** : à chaque itération, le temps écoulé depuis le dernier changement d'état (`tempsActuel - tempsPrecedent`) est comparé à la durée attendue pour l'état courant (`dureeEtape`, déterminée par une suite de `if/else if`). Cette approche, basée sur `millis()` plutôt que sur `delay()`, garantit que le programme reste disponible pour exécuter d'autres tâches (lecture de capteurs, gestion de boutons) en parallèle de la temporisation du feu – un point particulièrement pédagogique pour des débutants habitués à l'usage systématique de `delay()`.
4. **Transition d'état (switch) et fonction utilitaire (rafraichirLeds)** : lorsque la durée de l'état courant est écoulée, un `switch` détermine l'état suivant selon la séquence

cyclique Vert → Jaune → Rouge → Vert, et met à jour les sorties via la fonction `rafraichirLeds(v, j, r)`, qui centralise les trois appels à `digitalWrite()` en une seule fonction lisible et réutilisable.

Points forts du code :

- Utilisation de `millis()` plutôt que de `delay()`, une excellente pratique dès ce niveau d'introduction, qui évite de prendre de mauvaises habitudes de programmation bloquante.
- Modélisation explicite par `enum`, rendant le code auto-documenté et facilement compréhensible même par un lecteur découvrant le projet.
- Fonction utilitaire `rafraichirLeds()` qui factorise la commande des trois sorties, limitant la duplication de code et les risques d'erreur.
- Constantes de durée clairement nommées (`DUREE_VERT`, `DUREE_JAUNE`, `DUREE_ROUGE`), facilitant tout ajustement du minutage sans toucher à la logique du programme.

Pistes d'amélioration possibles :

- Ajouter une phase de transition « rouge-jaune » simultanée (comme dans certains systèmes de signalisation réels) avant le retour au vert, en introduisant un état supplémentaire dans l'`enum`.
- Généraliser le code à plusieurs feux synchronisés (carrefour à deux voies perpendiculaires), où le feu vert d'un axe correspond au feu rouge de l'autre – exercice naturel d'extension de la machine à états actuelle.
- Remplacer la suite de `if/else` `if` déterminant `dureeEtape` par un tableau indexé sur l'état (`unsigned long durees[] = {DUREE_VERT, DUREE_JAUNE, DUREE_ROUGE}`), pour un code légèrement plus compact et extensible.
- Ajouter un mode clignotant (LED jaune clignotante seule) simulant un feu en mode « avertissement », activable par exemple via un bouton poussoir.

8 Ressources Complémentaires

- **Vidéo** – *Arduino Basics : Digital Output (GreatScott!)*
https://www.youtube.com/watch?v=fL0BAO_Gd5M

9 Conclusion

Ce projet, volontairement simple, permet d'acquérir des réflexes fondamentaux de la programmation Arduino : commande de sorties numériques, dimensionnement de résistances limitatrices de courant, et surtout la maîtrise de la **gestion du temps non bloquante** via `millis()`. La structuration en machine à états, bien qu'appliquée ici à seulement trois phases, constitue une compétence directement transposable à des projets bien plus complexes (systèmes d'alarme, menus interactifs, protocoles de communication), faisant de ce feu de signalisation un excellent exercice de fondation avant d'aborder des projets Arduino plus avancés.